

싸피증권 서비스 포팅가이드

1. 개요 (Overview)

본 문서는 14기 특화 PJT 싸피증권(A503 팀) 서비스의 인프라 구성, 배포 방식 및 운영 전략을 정의한 문서이다.

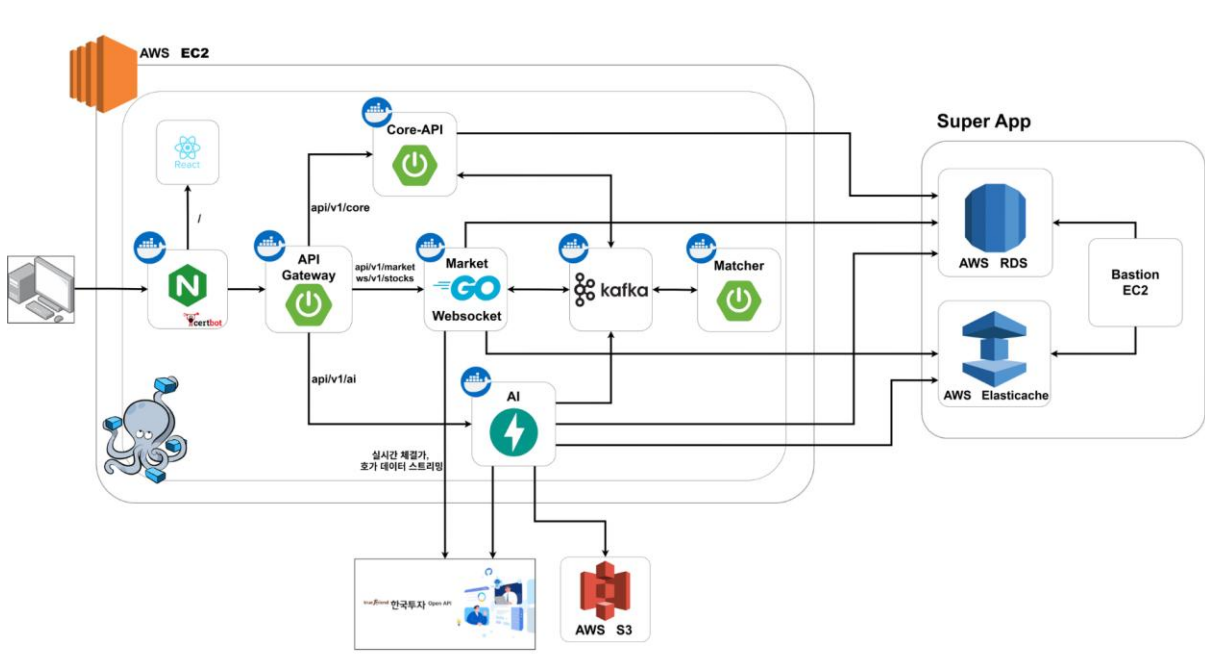
본 프로젝트는 실시간 주식 데이터 처리와 AI 기반 자동매매 기능을 제공하는 서비스로, 안정적인 운영과 빠른 배포를 목표로 한다.

A503 팀은 프로젝트 특성과 개발 기간을 고려하여 **Docker Compose** 기반의 경량 인프라 구조를 채택하였다.

1.1 목표

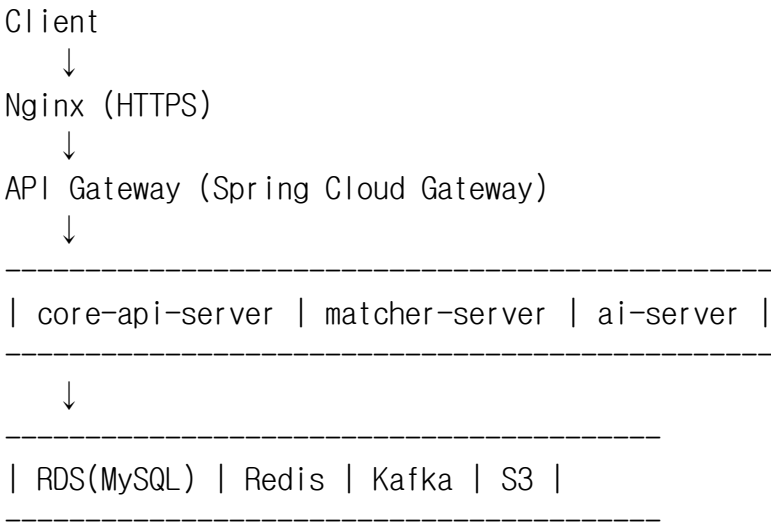
- 서비스 안정성 확보 (KIS API 실시간 데이터 처리 및 주문 시스템)
- 내부 원장 DB를 이용한 체결엔진 구현
- CI/CD 기반 자동 배포 환경 구축
- Kafka 기반 이벤트 처리 구조 확립
- Redis 및 DB 기반 데이터 일관성 유지

2. 시스템 아키텍처 (System Architecture)



2.1 전체 구조

싸피증권 서비스는 단일 진입점과 서비스 분리 구조를 기반으로 구성된다.



2.2 아키텍처 특징

- **Nginx**: HTTPS 처리 및 Reverse Proxy 역할 수행
- **API Gateway**: 인증(JWT) 및 서비스 라우팅 담당
- **Core API Server**: 사용자, 계좌, 주문 등 주요 비즈니스 로직 처리
- **Matcher Server**: 주문 매칭 및 체결 처리
- **AI Server (FastAPI)**: 자동매매 전략 생성 및 판단
- **Kafka**: 서비스 간 비동기 이벤트 처리
- **Redis**: 캐싱 및 Refresh Token 관리
- **RDS(MySQL)**: 주요 데이터 저장
- **S3(storage)**: AI 전략 json data 및 이미지 파일 저장

3. 인프라 구성 (Infrastructure Layer)

3.1 서비스 구성 및 포트 정보

싸피증권 서비스는 Docker Compose 기반으로 관리되며, 각 서비스는 stock-network 외부 네트워크를 통해 통신한다. 현재 구성된 서비스와 포트는 다음과 같다.

서비스명	내부 포트	외부 포트	역할
api-gateway	8080	8080	API Gateway, 인증 및 라우팅

서비스명	내부 포트	외부 포트	역할
core-api-server	8081	8081	사용자, 계좌, 주문, 알림 등 핵심 비즈니스 로직
matcher-server	8082	8082	주문 매칭 및 체결 처리
market-server	8085	8085	시장 데이터 및 종목 정보 처리
ai-server	8086	8086	AI 전략 분석 및 자동매매 판단

각 서비스는 독립된 컨테이너로 실행되며, 장애 발생 시 개별 재시작 및 재배포가 가능하도록 구성되어 있다.

3.2 네트워크 구성

모든 애플리케이션 서비스는 Docker 외부 네트워크인 stock-network 를 사용한다.

이를 통해 컨테이너 간 통신 시 서비스명을 호스트처럼 사용할 수 있으며, 예를 들어 api-gateway:8080, core-api-server:8081, matcher-server:8082, market-server:8085, ai-server:8086 형태로 내부 접근이 가능하다.

이와 같은 구성은 서비스 간 연결을 단순화하고, Docker Compose 환경에서의 운영 편의성을 높인다.

3.3 데이터 베이스 구성

본 프로젝트는 단순성과 운영 안정성을 고려하여 RDB 중심 구조를 채택하였다.

구성 요소	역할
RDS (MySQL)	사용자, 주문, 계좌 등 주요 데이터 저장
Redis	세션 관리 및 캐싱, Refresh Token 저장
ChromaDB	AI 임베딩 데이터 저장
S3	AI 전략 json data, 이미지(logo) 저장

3.4 메시지 브로커 (Kafka)

Kafka 는 서비스 간 비동기 통신을 위한 핵심 요소로 사용된다.

주요 역할

- 주문 이벤트 전달
- 체결 이벤트 처리

- AI 전략 실행 이벤트 전달

설정

- INTERNAL: kafka:29092 (컨테이너 내부 통신)
- EXTERNAL: EC2_PUBLIC_IP:9092 (외부 접근)

현재 AI Server 는 /app/chroma_db 경로를 사용하도록 설정되어 있으며, 실제 서버에서는 /home/ubuntu/ai-server-data/chroma_db 볼륨과 연결되어 데이터가 유지되도록 구성되어 있다.

3.5 볼륨 구성

일부 서비스는 데이터 영속성을 위해 Docker 볼륨 또는 호스트 디렉토리를 사용한다.

- matcher-server 는 /var/lib/matcher-data 경로를 사용하여 체결 관련 데이터 및 운영 로그를 저장한다.
- ai-server 는 /home/ubuntu/ai-server-data/chroma_db 를 /app/chroma_db 에 마운트하여 AI 관련 데이터가 컨테이너 재시작 이후에도 유지되도록 구성한다.

4. 애플리케이션 구성 (Application Layer)

4.1 API Gateway

API Gateway 는 외부 요청을 내부 서비스로 전달하는 관문 역할을 수행한다. 주요 기능은 다음과 같다.

- JWT 기반 인증 필터 처리
- 서비스별 경로 라우팅
- CORS 설정 적용
- 외부 요청의 공통 진입점 제공

API Gateway 는 8080 포트를 사용하며, 프로덕션 환경에서는 SPRING_PROFILES_ACTIVE=prod 로 실행된다. 또한 AUTH_CORS_ALLOWED_ORIGINS 환경 변수를 통해 허용 Origin 을 관리한다.

라우팅 구조

/api/v1/core → core-api-server

/api/v1/ai → ai-server

/api/v1/market → market-server

4.2 Core API Server

Core API Server 는 서비스의 핵심 비즈니스 로직을 담당하는 백엔드 서버이다. 주요 기능은 다음과 같다.

- 회원 관리
- 계좌 관리
- 주문 생성 및 조회
- 알림 처리
- 인증 관련 로직 연계

Core API Server 는 8081 포트를 사용하며, API Gateway 뒤에서 동작한다. 프로덕션 프로파일을 적용하여 실행되며, 인증 및 프론트엔드 연동을 위해 CORS 관련 환경 변수도 함께 주입된다.

4.3 Matcher Server

Matcher Server 는 주문 매칭 및 체결 엔진 역할을 수행한다. 사용자가 생성한 주문 정보를 바탕으로 매수/매도 매칭을 수행하며, 체결 데이터를 생성하는 핵심 서비스이다.

Matcher Server 는 8082 포트를 사용하며, /var/lib/matcher-data 볼륨을 통해 운영 중 생성되는 데이터와 로그를 보존한다.

이 서비스는 실시간 거래 처리 특성상 안정성과 데이터 유지가 중요한 구성 요소이다.

4.4 Market Server

Market Server 는 주식 종목 정보, 시세, 캔들 데이터 등 시장 관련 기능을 담당하는 서비스이다.

실시간 시세 연동, 종목 조회, 차트 데이터 제공 등의 역할을 수행하며, 8085 포트를 사용한다.

이 서비스는 프론트엔드에서 종목 검색, 시세 조회, 차트 조회 기능을 사용할 때 핵심적으로 연결된다.

4.5 AI Server

AI Server 는 FastAPI 기반으로 구현된 서비스로, 자동매매 전략 생성, 뉴스 분석, 투자 판단 등의 AI 기능을 담당한다.

8086 포트를 사용하며, AI_SERVER_CHROMA_DIR=/app/chroma_db 환경 변수를 통해 내부 저장 경로를 지정한다.

또한 호스트 서버의 /home/ubuntu/ai-server-data/chroma_db 디렉토리를 컨테이너 내부 /app/chroma_db 와 연결하여 AI 데이터가 영속적으로 유지되도록 구성하였다.

5. 네트워크 및 보안 전략

5.1 HTTPS 및 Reverse Proxy

- Nginx 를 통해 HTTPS 적용
- Let's Encrypt 기반 인증서 사용

Client → HTTPS → Nginx → API Gateway → Backend

5.2 인증 및 보안

인증 및 인가 처리는 API Gateway 를 중심으로 수행한다.

JWT 기반 인증 구조를 적용하였으며, 토큰 검증 후 인증이 필요한 요청만 내부 서비스로 전달한다. Refresh Token 은 Redis 에 저장하여 세션 관리와 보안성을 함께 확보하였다.

- JWT (RS256) 기반 인증
 - Refresh Token Redis 저장
 - Gateway 에서 인증 필터 처리
-

6 시크릿 및 환경 변수 관리 (Secret & Environment Configuration)

싸피증권(A503) 서비스는 운영 환경에서 필요한 모든 설정 값을 .env 파일 기반으로 관리하며, 배포 시 Jenkins Credentials 를 통해 주입하는 방식으로 보안을 유지한다.

기존 Kubernetes 기반 구조에서는 Secret 리소스를 통해 관리하였으나 , A503 팀은 Docker Compose 환경에 맞게 **환경 변수 기반 설정 방식**을 적용하였다.

5.1 공통 환경 설정 (Common Configuration)

서비스 실행 환경에 대한 기본 설정은 다음과 같다.

항목	값	설명
SPRING_PROFILES_ACTIVE	prod	운영 환경 프로파일
TZ	Asia/Seoul	서버 시간 설정

5.2 데이터베이스 설정 (Database Configuration)

MySQL 기반 RDS 를 사용하여 주요 데이터를 관리한다.

항목	값
DB_HOST	rds-stg.ssafyapp.com
DB_PORT	3306
DB_NAME	trader26
DB_USER	trader26
DB_PASSWORD	(비공개)
DB_URL	jdbc:mysql://rds-stg.ssafyapp.com:3306/trader26

특징

- AWS RDS 기반 관리형 데이터베이스 사용
- timezone 및 encoding 설정 포함
- JDBC URL 기반 연결 방식 사용
- SUPER APP 선정으로 인한 SSH 터널링 접속 방법

5.3 Redis 설정 (Cache & Session)

Redis 는 캐싱 및 인증 토큰 관리를 위해 사용된다.

항목	값
REDIS_HOST	redis-stg.ssafyapp.com
REDIS_PORT	6379
REDIS_PASSWORD	(없음)

역할

- Refresh Token 저장
- 인증 세션 관리
- 일부 캐시 데이터 처리
- SUPER APP 선정으로 인한 SSH 터널링 접속 방법

5.4 인증 및 보안 설정 (Authentication & Security)

JWT 기반 인증을 적용하며, 공개키/개인키 방식(RS256)을 사용한다.

항목	설명
JWT_SECRET_KEY	토큰 서명용 키
JWT_PRIVATE_KEY_PATH	개인키 경로
JWT_PUBLIC_KEY_PATH	공개키 경로

항목	설명
JWT_ISSUER	토큰 발급자
JWT_ACCESS_EXPIRATION	Access Token 만료 시간
JWT_REFRESH_EXPIRATION	Refresh Token 만료 시간

특징

- RS256 기반 서명 방식
- Gateway 에서 인증 처리
- Redis 기반 Refresh Token 관리

5.5 CORS 설정

프론트엔드 접근 허용을 위해 CORS 정책을 설정한다.

항목	값
AUTH_CORS_ALLOWED_ORIGINS	https://j14a503.p.ssafy.io , http://localhost:5173

5.6 Kafka 설정 (Event Streaming)

Kafka 는 서비스 간 비동기 이벤트 처리를 위해 사용된다.

항목	값
KAFKA_BOOTSTRAP_SERVERS	kafka:29092
KAFKA_BROKER	9092
KAFKA_CLUSTER_ID	MkU3OEVBNtcwNTJENDM2Qk
KAFKA_TICK_TOPIC	market.tick

항목	값
KAFKA_ORDERBOOK_TOPIC	market.orderbook

특징

- INTERNAL 네트워크 기반 (kafka:29092)
- 이벤트 기반 구조 (시세, 호가, 체결 등)

5.7 AWS 및 S3 설정 (Storage)

AWS S3 를 활용하여 파일 및 데이터 저장을 수행한다.

항목	값
S3_BUCKET	ssafyapp-stg-project-prv-data
S3_REGION	ap-northeast-2
S3_PREFIX	tlu600/

활용

- 종목 로고 이미지 저장
- 데이터 파일 관리
- AI 관련 리소스 저장

5.8 외부 API 설정 (KIS API)

한국투자증권(KIS) API 를 활용하여 실시간 주식 데이터를 수신한다.

항목	값
KIS_BASE_URL	https://openapivts.koreainvestment.com:29443

항목	값
KIS_WS_URL	ws://ops.koreainvestment.com:31000

특징

- REST + WebSocket 혼합 사용
- 실시간 시세 및 호가 데이터 처리

5.9 AI 서버 전용 설정

AI Server 에서 사용하는 별도 설정이다.

항목	설명
AI_KIS_APP_KEY	AI 전용 API Key
AI_KIS_APP_SECRET	AI 전용 Secret
AI_SERVER_CHROMA_DIR	벡터 DB 저장 경로

5.10 내부 서비스 연동 설정

서비스 간 통신을 위한 내부 URL 설정이다.

항목	값
CORE_API_URL	http://core-api-server:8081/api/v1
AUTO_TRADE_USER_ID	1

특징

- Docker 네트워크 기반 내부 호출
- API Gateway 를 거치지 않고 내부 직접 통신

5.11 기타 설정

항목	값	설명
GMS_API_KEY	내부 서비스 인증 키	
USE_SSH	false	SSH 터널링 여부

5.12 보안 관리 전략

- .env 파일은 Git 에 포함하지 않음
 - Jenkins Credentials 를 통해 배포 시 주입
 - 서비스별 환경 변수는 공통 .env 로 관리
-

6. 배포 및 운영 전략 (Deployment & Ops)

6.1 CI/CD 파이프라인

싸피증권 서비스는 Jenkins 기반 CI/CD 구조를 사용한다.

배포 흐름

1. 개발자가 Git(dev)에 코드 Push
 2. Jenkins 빌드 트리거
 3. Docker 이미지 빌드
 4. docker-compose 기반 서비스 재배포
-

6.2 서비스 배포 방식

변경된 서비스만 선택적으로 재배포한다.

```
docker-compose stop api-gateway
docker-compose rm -f api-gateway
docker-compose up -d --build api-gateway
```

특징

- 빠른 배포
- 서비스 중단 최소화
- 효율적인 리소스 사용

6.3 이미지 관리

각 서비스는 개별 Dockerfile 을 통해 관리된다.

- core-api-server
- api-gateway
- ai-server
- matcher-server

7. 운영 및 모니터링 전략 (Monitoring & Health Check)

싸피증권(A503)은 안정적인 서비스 운영을 위해 모니터링 시스템과 헬스 체크 기능을 구축하였다. 이를 통해 시스템 상태를 실시간으로 확인하고, 장애 발생 시 빠르게 대응할 수 있도록 설계하였다.

7.1 모니터링 시스템 구축

싸피증권 서비스는 Prometheus 와 Grafana 를 활용하여 시스템 및 애플리케이션 상태를 실시간으로 모니터링한다.

구성

- Prometheus:** 메트릭 수집 및 저장
- Grafana:** 시각화 대시보드 제공

주요 모니터링 항목

- CPU 및 메모리 사용량
- 컨테이너 상태
- 서비스 응답 시간

- API 호출 수 및 에러율

효과

- 서비스 이상 징후 조기 감지
 - 운영 상태 시각화
 - 장애 대응 시간 단축
-

7.2 헬스 체크 기능

서비스의 정상 동작 여부를 확인하기 위해 각 서버에 헬스 체크 기능을 구현하였다.

Spring Boot (Core API, Gateway 등)

- Spring Boot Actuator 를 활용하여 헬스 체크 엔드포인트 제공
- `/actuator/health` 경로를 통해 서비스 상태 확인 가능

FastAPI (AI Server)

- `/health` endpoint 구현
- 서비스 정상 동작 여부 확인

효과

- 서비스 상태 자동 점검
 - 배포 및 재시작 시 안정성 확보
 - 장애 발생 시 빠른 원인 파악
-

8 향후 개선 과제 (Recommendations)

현재 시스템 안정성 및 확장성을 위해 다음 개선이 필요하다.

8.1 Kafka 안정성 개선

- 향후 Replica 구성 필요
-

8.2 데이터 안정성 강화

- RDS 백업 정책 점검
 - 장애 대응 전략 수립
-

9. 결론

싸피증권(A503) 서비스는 Docker 기반의 경량 인프라 구조를 통해 빠른 개발 및 배포 환경을 구축하였다.

Kafka 기반 이벤트 처리와 AI 자동매매 시스템을 중심으로 확장성과 안정성을 동시에 고려한 구조를 설계하였으며, 향후 기존 100 종목에서 전체 상장종목 및 해외주식으로까지 확장하고, 체결엔진 및 시세 서버 사용량 증가에 따른 최적화 고도화를 통해 완성도를 높일 예정이다.